

FitTrack, a Social Fitness System: Deduced to Machine Level, Then Cut to the Bone

Live activity feed - Personal top activities - Leaderboards: a three-table minimal architecture, fully deduced (V2 + full journey)

Methodology: Features -> Data Flow -> Scale -> Failure -> KPI, in two passes

1 - Problem and Scale Anchors

Design FitTrack, a social fitness app: **R1** followers see a user's near-real-time progress while working out (location / pace / distance); **R2** a user queries their top activities for a day / month / past year; **R3** weekly distance leaderboards, global and among friends.

Scale: 50M registered / 5M DAU / peak **1M concurrent workouts**, one telemetry point per phone every 5s; events arrive late, out of order, duplicated; celebrity accounts have millions of followers.

Two anchor lines (the ledger for every number):

1) Problem scale: 50M / 5M DAU / 1M concurrent; 2) Capacity sized against a **1B-user upper bound** (~200x headroom). Declare the ledger before switching.

Assumed parameters: feed refresh 10s; friends capped at 1000.

2 - Methodology: Layered Waterfall x Two Passes

- **Waterfall:** Features -> Data Flow -> QPS/Scale -> Failure -> KPI — contract first, then flow, then numbers, then failure stories;
- **Two passes:** pass one sweeps the end-to-end data flow in minutes (all three requirements' exits named — completeness locked first); pass two revisits each component with capacity numbers and failure handling. Even if pass two is cut short, coverage is already banked;
- This note also records what got **cut** (ch.9) — half of design is building, the other half is deleting.

3 - Features (the contract)

- **Two-tier UI:** newsfeed list (one snippet per running friend: running? / elapsed / distance / **neighborhood-level location**) -> tap into a detail page (full trajectory, pace). Rationale: payload and QPS tiering;
- **Visible only while running:** liveness is data-driven — telemetry keeps arriving = running; 20s without data = expired (TTL semantics), no reliance on an end event;
- **Privacy:** coarse location + per-user toggle; trajectory retains only the last ~10 minutes (retention policy doubles as the privacy window, ch.7);
- **Viewer offline:** cached last state, labeled 'updated X min ago';
- **Reverse geocoding:** on-device resolution (coarse, zero server cost) primary; server-side distance-threshold + geohash-cell cache as fallback;
- Likes/comments deferred to v2.

4 - Pass One: End-to-End Data Flow (one spine, three exits)

client reports {userid, eventid, ts, gps, district_id, type} (~50B) every 5s (eventid unique per workout = userid + random; reconnect rule: same eventid -> splice incrementally, different -> full refresh)

-> **ingestion/log tier** (append to disk, keyed by ts, **source of truth**; receive + store + process in one tier)

-> update stream fans into three consumers:

1) **Snippet table** (userid -> latest state) = R1 list exit;

2) **Tracking table** (userid -> last 100 points, sliding window) = R1 detail exit;

3) **Offline derivation:** activity summary rows (one per finished workout) = R2 exit; summaries roll into weekly totals = R3 exit (global batch top-100 / friends multiget + sort).

All three requirements placed — coverage locked; pass two begins.

5 - Pass Two, Part 1: Ingestion/Log Tier (4-6 machines, all-in-one)

- Write QPS: **1M concurrent / 5s = 200K events/s peak** (rule: QPS = concurrency / interval); mean 10-20K/s, peak-to-mean ~10:1;
- Bandwidth: $200K \times 50B = 10MB/s$ — one machine's throughput; **2-3 shards x primary/replica exist for blast radius and downstream parallelism, not throughput**; local disk against power loss;
- Volume: 3 months of raw ~ 78TB (compressed ~1/5), <1PB per decade — raw must be kept: the only source of truth for debug / recovery / offline stats; every derived table is recomputable;
- Fluctuation: with independent phases, per-second arrivals ~ 200K +/- 400 (0.2%, Binomial); independence enforced by jitter (fixed period + random phase, mean preserved);
- **Storm handling**: a regional-outage reconnect burst exceeds 200K instantaneously — the log absorbs it as backlog; consumers provisioned at mean x2-3 drain it (worked example: 1-minute outage backlogs ~12M events; drained in ~30s at 2x — 'minute-scale storm, second-scale digestion');
- Failure: replica takes over + replay from offset; idempotent key (eventid, ts) makes replay safe.

6 - Part 2: Snippet Table — Full Replication, Zero Fan-out (the heart)

6.1 Structure and capacity

- One **latest-state** row per user: running bit (1B) + district id (8B) + speed (1B) + last-update ts (8B) + latest eventid (8B) ~ 28B, ~5B compressed;
- **28B x 1B users = 28GB — fits in one machine's memory** -> fully replicate to ~20 machines, one complete copy each;
- **Presence dissolved**: 'is she running' is just the state bit judged by last-update ts + a 20s window — no standalone presence service; one local multiget answers 'who is running' AND returns the snippets: two read steps merged into one.

6.2 The ledger, and the soul argument

- Reads: $5M \times (7200s \text{ online} / 10s \text{ refresh}) = 3.6B \text{ req/day} / 86400 \sim \mathbf{40K \text{ QPS mean} \rightarrow x10 \text{ envelope} = 400K \text{ peak} \rightarrow 20K/machine}$; the friend list (≤ 1000 ids) probes a local hash table in 50-100 microseconds — **zero cross-machine fan-out**;
- Writes: each replica subscribes to the update stream and batch-applies 200K/s (replica-pull sync; nobody pushes a fan-out);
- **The soul argument: write amplification x20 buys away ALL read-side fan-out — reads (400K) > writes (200K), so the tax is worth paying.** The essence of full replication is converting read-heavy into acceptable write-heavy.

6.3 Tail governance: caps guard the tail, expectations size the fleet

- Product cap: show at most 20-100 running friends (sorted by start time) — the UI can't show 1000 anyway; product truncation = natural backpressure;
- Expected load: running ratio $1M/50M = 2\%$, median 300 friends -> ~6 running friends — payload reads are tiny;
- Worst case (everyone at cap): $400K \times 100 = 40M \text{ probes/s}$ over the snapshot fleet = ~280K/machine — still inside single-node in-memory lookup capacity. **Both bounds sealed: worst case sets fuses; expectation sizes the fleet.**

7 - Part 3: Tracking Table — Sharded; the Sliding Window IS the Privacy Window

- Last **100 trajectory points** per user (x 32B ~ 3.2KB): at 5s per point the window is ~8-10 minutes — 'keep only the last 100 points' solves bounded storage and the privacy cap in one sentence;

- Capacity: active ledger 1M x 3.2KB ~ 3.2GB; 1B upper bound 3.2TB / 20 shards ~ 160GB/machine — **does not fit one machine, hence sharding**;
- Writes: 200K points/s / 20 shards = 10K ring-writes/machine (fixed ring, new overwrites old, microseconds);
- Reads: detail page is tap-through only, one-two orders lower QPS; **single-key (userid) hash goes straight to its shard — fan-out only exists on 'query many at once' paths; single-key lookups are immune**;
- Older history goes through the low-frequency raw-archive path.

The contrast between Parts 2 and 3 is the core idiom of this design: replicate the thin, shard the fat. Small, globally-probed tables (snippet) buy away fan-out via full replication; large, point-queried tables (tracking) spread capacity via sharding.

8 - Part 4: Offline Derivation — R2 / R3

- **Activity summary table**: one row per finished workout {distance/duration/date/...}; write rate = completion rate, only ~1K/s peak; key = userid partition + ts sort -> 'last 100 workouts of a user' = a contiguous range scan on one shard, milliseconds;
- **R2 (top activities)**: a person logs a few hundred rows a year — **compute on read; rollup pre-aggregation is over-engineering at this size** (say the trade-off aloud); random inserts are absorbed into sequential writes by LSM/memtables;
- **R3 (leaderboards), twin tracks**: global = hourly batch top-100 -> cache, direct reads (precompute side); friends = client-held list -> multiget weekly totals -> sort in place (1000 entries < 0.1ms, compute-on-read side) — **the asymmetry mirrors the two queries' load profiles**;
- KPI in one line: per-tier counters (QPS/latency/backlog/watermarks) + Spark/Presto offline aggregation, read together with DAU/clicks.

9 - The Razor Log: Five Machines We Cut

Component cut	The ledger that killed it
Explicit dedup / sequence machinery	reads consume latest-state snapshots (naturally idempotent); write-side dedup folds into the (eventid, ts) idempotent-upsert key — machinery gone, semantics kept
Event-driven 'active friends list' subsystem	start/stop fan-out + per-viewer lists + reconciliation = four new appliances; dumb multiget against the replicated table already balances. Kept as an evolution reserve for second-level refresh
Bloom-filter existence checks	the active set is 8MB-28GB — small enough to replicate exactly; probabilistic structures exist for sets too big to copy, which is false here; 1% false positives would add wasted payload reads
Standalone presence service	a state bit + timestamp inside the snippet row is equivalent — 'merge when thin, split when fat'; split it back out if the row ever grows
Per-viewer inbox fan-out-on-write (Moments-style)	telemetry pushes 200K updates/s x followers -> push sentenced to death here. Verdict formula: push cost = post rate x followers; pull cost = refresh QPS x followees — in low-frequency posting the formula flips and push revives

10 - Evolution Reserves (four cards in the pocket)

- **Celebrities**: pull-based reads are naturally immune to million-follower accounts — no special path exists; a follower-count exemption only matters if write fan-out is ever introduced;

- **Moments/post variant:** the verdict formula flips (500 posts/s x ordinary friend counts -> fan-out-on-write viable), or stay pull with a time-slotted post index (10-min slots, ~6MB each, same full-replication idiom); storage = (userid, ts, postid) composite key, shard by userid, sort by ts in-shard;
- **Write-path upgrade:** if strict multi-consumer semantics are ever needed, the ingestion tier evolves into a standard durable log (properties: persisted, replayable, multi-reader, per-user ordered);
- **Snippet row grows fat:** split the liveness bit back into a small presence table — merging and splitting are functions of size.

11 - Verdict Proverbs (the whole note, compressed)

Proverb	One-line expansion
QPS = concurrency / interval	the only bridge from state to traffic
Unit discipline	suffix every number: /day vs /s; /machine vs /replica-group
Sanity check	compare any QPS to 'all of Google: ~1M-class'; exceeding it means a unit error
Budget = QPS x fan-out width x unit cost	fan-out is not a sin; the three-factor product is
Hot data small enough -> replicate fully	pay write amplification to buy away read fan-out; worth it when reads > writes
Replicate the thin, shard the fat	small global tables replicate; big point-query tables shard
Caps guard the tail; expectations size the fleet	worst case sets fuses, not machine counts
Keys grow out of queries; tables out of distribution strategy	schema is derived from access patterns
Raw is the source of truth	replayable log + idempotent keys = the escape hatch for every downstream failure
Pessimistic bounds prove feasibility; real numbers set budgets	state both
Name properties, not brands	'a persisted, replayable, multi-reader, per-user-ordered log'

Closing: at this scale the hot data is small enough to replicate fully — the simplest architecture yields the strongest read path. End-to-end: ~50-70 machines for the stated scale; every tier scales linearly toward the 1B bound.

Appendix - The Full Journey (from five tables to three)

Before the final form, the deduction traveled further. These chapters preserve the machine-level designs as they stood — not dead ends, but the ledgers behind every cut; the post-feed variant remains a complete standalone design.

A - Standalone Presence Service (later dissolved into the snippet table)

- Table: `userid` -> `last_active_ts` (8B) + active bit (double-buffered, whole-buffer flip every 5s — same idiom as base/delta); **update source pinned: only workout telemetry renews; viewer feed requests never do**;
- Capacity: problem ledger 50M x ~10B ~ 500MB; 1B bound 8-16GB — **fits one machine -> a full copy per node, 20 nodes = 20 replicas, zero cross-machine fan-out** (all <=1000 friend ids probed locally in 50-100 microseconds);
- Queries: 400K peak / 20 = 20K/machine; client sends its friend list (8KB) every 5s -> NIC ledger 3.2GB/s / 20 = 160MB/s/machine, within single-NIC capacity;
- Writes: update stream broadcast x20 replicas (320KB/s each — state the amplification aloud);
- **Why it dissolved:** the 28B snippet row already carries a state bit + timestamp — under the same full-replication premise a separate presence table became redundant. 'Merge when thin, split when fat.'

B - Variant Deduction: the Post Feed (Moments-ified), full machine level

Verdict formula: push cost = post rate x follower count; pull cost = refresh QPS x followee count.

Telemetry: 200K updates/s x followers -> push dies. Moments: 5M x ~10 posts/day ~ 500 writes/s (peak ~1K) -> push revives. **Same parts, flipped load profile, flipped verdict.**

- **Chosen route: pull + time-slotted post index** — a small fully-replicated (`userid`, `post_ts`) index in 10-minute slots: 50M posts/day / 144 slots ~ 350K x 16B ~ 5-6MB/slot, ~800MB/day — the presence idiom again: small enough to copy whole;
- Split-frequency polling: snippets every 5s, posts every 60s (per-viewer friend post rate ~0.1 QPS) -> 3-30K read QPS;
- Backfill: the client walks slots backwards ('who posted in this slot'), then the two-step read; 300K QPS sits inside verified capacity; **slot boundaries double as pagination cursors**; deep archaeology degrades to lazy per-friend pulls;
- Two skins of storage: **post table (`userid`, `ts`, `postid`) composite key** — shard by `userid` only (one person, one shard), sort by `ts` + `postid` in-shard so pagination scans sequentially; content ~1KB / ~200B compressed; likes/comments stored as ids, hydrated at the detail layer; capacity: 1B x 10/day x 200B x 10y ~ PB-class upper bound, real 5M DAU ~200x smaller;
- **Celebrity exemption:** >10K followers -> outbox-pull. The disease is not post frequency but **single-post blast radius** (one post x 1M followers = a million-append pulse: clog the worker, or throttle and deliver late — both bad);
- Three dividends of 'people don't post much': outboxes tiny (cacheable whole); inboxes bounded (~7KB/day/user, retention closes the ledger); no hot authors on the write side;
- The alternative (inbox fan-out-on-write) was also costed: workers batch appends per target shard, 300K appends/s / 30 shards = 10K/s/shard, ~65 machines total — reserved for when refresh drops to seconds;
- **Storage vs distribution are orthogonal:** the content table is the only truth; inbox/outbox are the materialization of a distribution verdict — 'tables grow out of the distribution strategy' — and both are rebuildable from content + graph.

C - Write-Side Storage and Processing, detailed ledger

- Raw log, 3 months ~ 78TB (10MB/s uncompressed; ~1/5 compressed): keeping 3 months fully in memory = 140 x 100GB x 3 replicas = **420 machines, buying 'any historical second instantly queryable'** — a stated trade-off; fallback = last 7 days in memory + SSD tiering (~30-40 machines);
- Shard by (userid + eventid) hash against skew; archive slots keyed by arrival time, serving indexed by (eventid, client_ts) — two orderings, two jobs;
- In-memory KV random writes taken at a **conservative floor of 10K/s/machine** (benchmarks 100K+/s); append-only higher — two ledgers, stated separately;
- Stream-processing pessimistic bound: 10ms/event x 32 cores = 3.2K/s/machine -> 60-machine ceiling; real cost microseconds -> a handful — 'pessimistic bounds prove feasibility, real numbers set budgets';
- User table can serve via **base/delta double-buffer + pointer flip** (snapshot isolation; cost = 2x memory, stated).

D - Failure Modes, summary table

Failure	Handling	Grounds
Regional outage -> reconnect storm	log absorbs; consumers at mean x2-3	1-min backlog ~12M events, drained ~30s at 2x
Stream processor crash / bad logic	replay from offset; (eventid, ts) idempotent	raw is truth; derived recomputable
Snippet replica down	any of 20 full copies serves	seconds-stale invisible under the 20s window
Ingestion power loss	local disk + redundancy	write path loses nothing
Runner's phone offline / dead	local batching + late upload; eventid splice rule; last-write-wins by client_ts (no distance jump-back); missing end -> TTL expiry	existing machinery, zero new parts
Viewer offline	cached state + 'updated X min ago'	honest degradation

Final line: three tables (log as truth / snippet fully replicated / tracking sharded) + offline derivation, ~50-70 machines end to end; the appendices are the ledgers behind every cut. Hot data small enough to replicate fully — simplest architecture, strongest read path.